```python
# Import pandas for data handling
import pandas as pd

# Import matplotlib for visualization
import matplotlib.pyplot as plt

# Import seaborn for advanced visualization
import seaborn as sns

# Load CSV dataset
df = pd.read_csv("Bank_Churn.csv")

# Display first 5 rows
df.head()
```

```
   CustomerId   Surname  CreditScore Geography  Gender  Age  Tenure  \
0    15634602  Hargrave          619    France  Female   42       2
1    15647311      Hill          608     Spain  Female   41       1
2    15619304      Onio          502    France  Female   42       8
3    15701354      Boni          699    France  Female   39       1
4    15737888  Mitchell          850     Spain  Female   43       2

     Balance  NumOfProducts  HasCrCard  IsActiveMember
EstimatedSalary  \
0       0.00              1          1               1
101348.88
1   83807.86              1          0               1
112542.58
2  159660.80              3          1               0
113931.57
3       0.00              2          0               0
93826.63
4  125510.82              1          1               1
79084.10

   Exited
0       1
1       0
2       1
3       0
4       0
```

```python
# Check total rows and columns
df.shape
```

```
(10000, 13)
```

```python
# Display column names
df.columns
```

```
Index(['CustomerId', 'Surname', 'CreditScore', 'Geography', 'Gender',
'Age',
       'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
'IsActiveMember',
       'EstimatedSalary', 'Exited'],
      dtype='object')
```

```
# Check dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerId       10000 non-null  int64
 1   Surname          10000 non-null  object
 2   CreditScore      10000 non-null  int64
 3   Geography        10000 non-null  object
 4   Gender           10000 non-null  object
 5   Age              10000 non-null  int64
 6   Tenure           10000 non-null  int64
 7   Balance          10000 non-null  float64
 8   NumOfProducts    10000 non-null  int64
 9   HasCrCard        10000 non-null  int64
 10  IsActiveMember   10000 non-null  int64
 11  EstimatedSalary  10000 non-null  float64
 12  Exited           10000 non-null  int64
dtypes: float64(2), int64(8), object(3)
memory usage: 1015.8+ KB
```

```
# Statistical summary
df.describe()
```

```
         CustomerId   CreditScore           Age        Tenure
Balance  \
count  1.000000e+04  10000.000000  10000.000000  10000.000000
10000.000000
mean   1.569094e+07    650.528800     38.921800      5.012800
76485.889288
std    7.193619e+04     96.653299     10.487806      2.892174
62397.405202
min    1.556570e+07    350.000000     18.000000      0.000000
0.000000
25%    1.562853e+07    584.000000     32.000000      3.000000
0.000000
50%    1.569074e+07    652.000000     37.000000      5.000000
97198.540000
75%    1.575323e+07    718.000000     44.000000      7.000000
127644.240000
```

```
max    1.581569e+07    850.000000     92.000000     10.000000
250898.090000

       NumOfProducts    HasCrCard  IsActiveMember  EstimatedSalary  \
count   10000.000000  10000.00000    10000.000000     10000.000000
mean        1.530200      0.70550        0.515100    100090.239881
std         0.581654      0.45584        0.499797     57510.492818
min         1.000000      0.00000        0.000000        11.580000
25%         1.000000      0.00000        0.000000     51002.110000
50%         1.000000      1.00000        1.000000    100193.915000
75%         2.000000      1.00000        1.000000    149388.247500
max         4.000000      1.00000        1.000000    199992.480000

              Exited
count  10000.000000
mean       0.203700
std        0.402769
min        0.000000
25%        0.000000
50%        0.000000
75%        0.000000
max        1.000000
```

```
# Check null values in dataset
df.isnull().sum()
```

```
CustomerId         0
Surname            0
CreditScore        0
Geography          0
Gender             0
Age                0
Tenure             0
Balance            0
NumOfProducts      0
HasCrCard          0
IsActiveMember     0
EstimatedSalary    0
Exited             0
dtype: int64
```

```
# Remove null values
df.dropna(inplace=True)

# Count duplicate rows
df.duplicated().sum()
```

```
np.int64(0)
```

```
# Remove duplicate rows
df.drop_duplicates(inplace=True)
```

```
# Display data types
df.dtypes
```

```
CustomerId           int64
Surname             object
CreditScore          int64
Geography           object
Gender              object
Age                  int64
Tenure               int64
Balance            float64
NumOfProducts        int64
HasCrCard            int64
IsActiveMember       int64
EstimatedSalary    float64
Exited               int64
dtype: object
```

```
# Display cleaned dataset
df.head()
```

```
   CustomerId   Surname  CreditScore Geography  Gender  Age  Tenure  \
0    15634602  Hargrave          619    France  Female   42       2
1    15647311      Hill          608     Spain  Female   41       1
2    15619304      Onio          502    France  Female   42       8
3    15701354      Boni          699    France  Female   39       1
4    15737888  Mitchell          850     Spain  Female   43       2

     Balance  NumOfProducts  HasCrCard  IsActiveMember
EstimatedSalary  \
0       0.00              1          1               1
101348.88
1   83807.86              1          0               1
112542.58
2  159660.80              3          1               0
113931.57
3       0.00              2          0               0
93826.63
4  125510.82              1          1               1
79084.10

   Exited
0       1
1       0
2       1
3       0
4       0
```

```
# Count total customers
len(df)
```

```
10000
```

```
# Count churned and non-churned customers
df['Exited'].value_counts()
```

```
Exited
0    7963
1    2037
Name: count, dtype: int64
```

```
# Calculate churn percentage
churn_rate = df['Exited'].mean() * 100

# Print churn rate
print("Churn Rate:", round(churn_rate,2), "%")
```

```
Churn Rate: 20.37 %
```

```
# Calculate churn rate by country
country_churn = df.groupby('Geography')['Exited'].mean()*100

# Display result
country_churn
```

```
Geography
France     16.154767
Germany    32.443204
Spain      16.673395
Name: Exited, dtype: float64
```

```
# Calculate churn rate by gender
gender_churn = df.groupby('Gender')['Exited'].mean()*100

# Display result
gender_churn
```

```
Gender
Female    25.071539
Male      16.455928
Name: Exited, dtype: float64
```

```
# Compare active and inactive customers
activity = df.groupby('IsActiveMember')['Exited'].mean()*100

# Display churn percentage
activity
```

```
IsActiveMember
0    26.850897
1    14.269074
Name: Exited, dtype: float64
```

```
# Calculate average balance
df['Balance'].mean()
```

```
np.float64(76485.889288)
```

```
# Calculate average salary
df['EstimatedSalary'].mean()
```

```
np.float64(100090.239881)
```

```
 # Calculate average credit score
df['CreditScore'].mean()
```

```
np.float64(650.5288)
```

```
# Churn rate by number of products
products = df.groupby('NumOfProducts')['Exited'].mean() * 100

# Display result
products
```

```
NumOfProducts
1     27.714398
2      7.581699
3     82.706767
4    100.000000
Name: Exited, dtype: float64
```

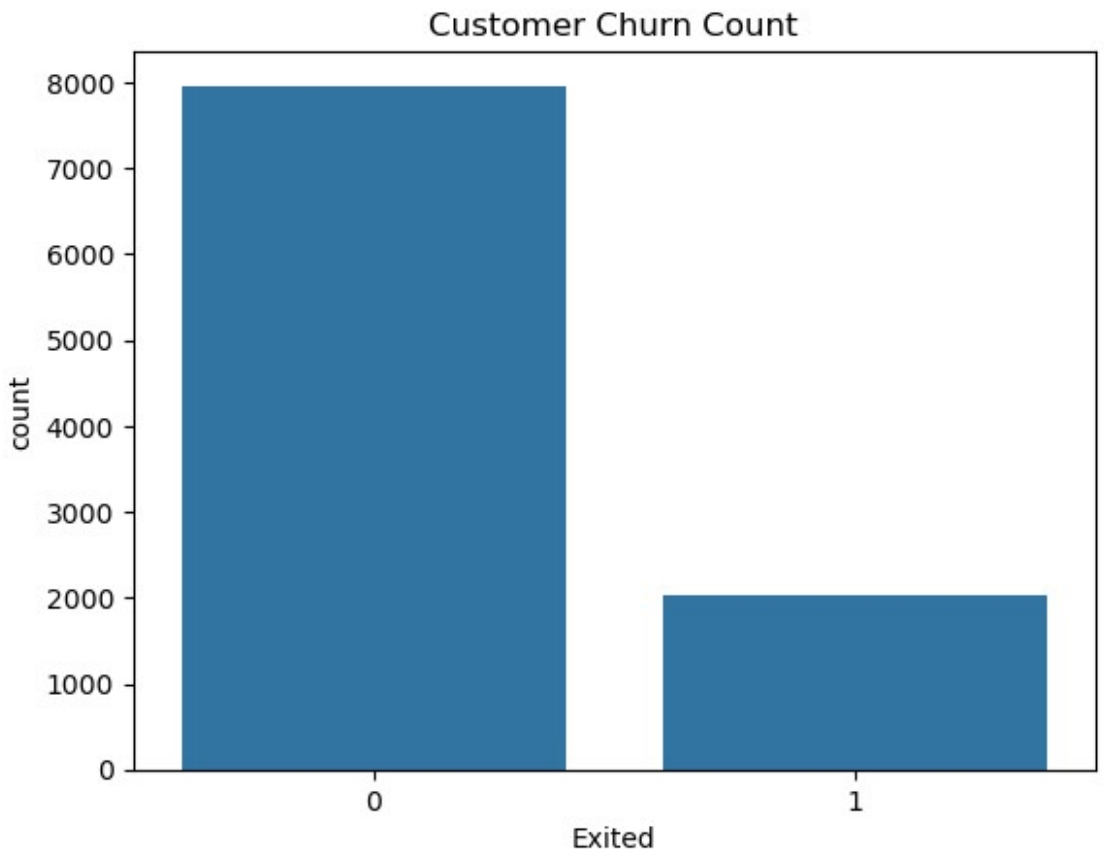
```
# Create age groups
df['AgeGroup'] = pd.cut(
    df['Age'],
    bins=[18,30,40,50,60,100],
    labels=['18-30','31-40','41-50','51-60','60+']
)

# Display age groups
df['AgeGroup'].value_counts()
```

```
AgeGroup
31-40    4451
41-50    2320
18-30    1946
51-60     797
60+       464
Name: count, dtype: int64
```

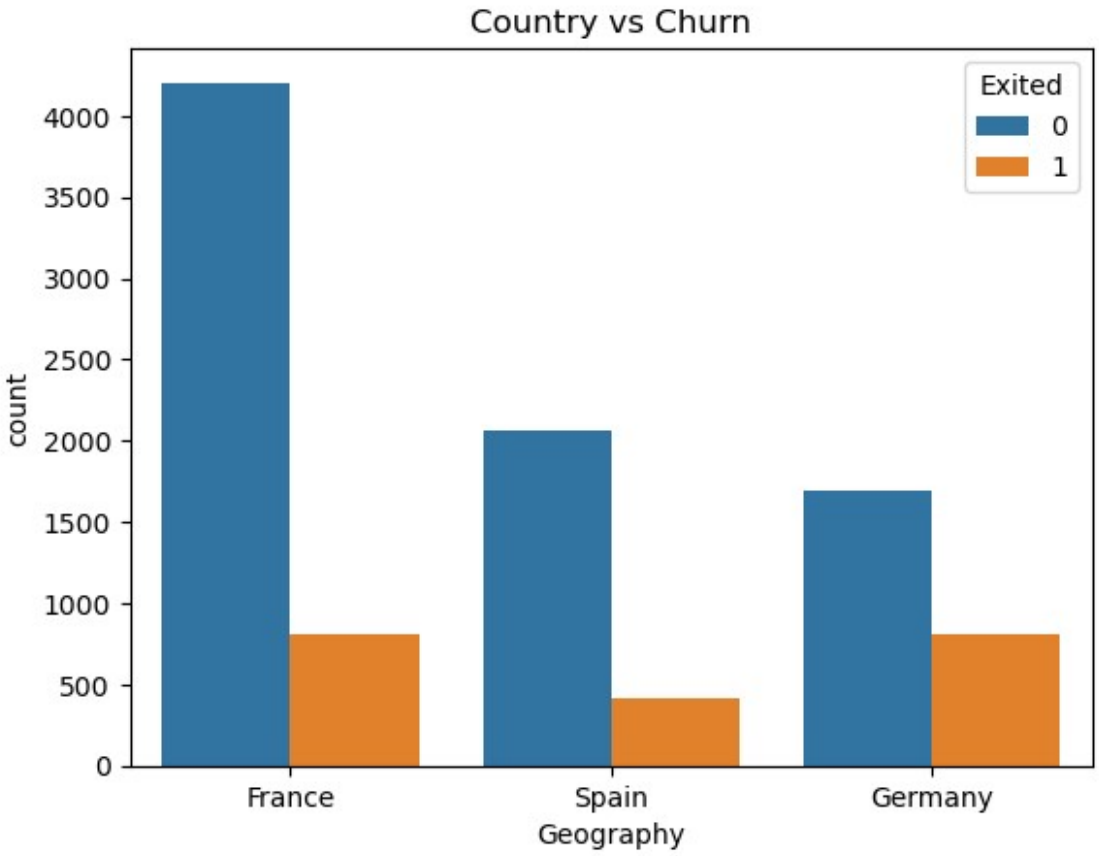
```
# Calculate churn by age group
age_churn = df.groupby('AgeGroup')['Exited'].mean() * 100

# Display result
age_churn
```

```
C:\Users\abhij\AppData\Local\Temp\ipykernel_32604\2887426137.py:2:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.
  age_churn = df.groupby('AgeGroup')['Exited'].mean() * 100

AgeGroup
18-30     7.502569
31-40    12.087171
41-50    33.965517
51-60    56.210790
60+      24.784483
Name: Exited, dtype: float64
```

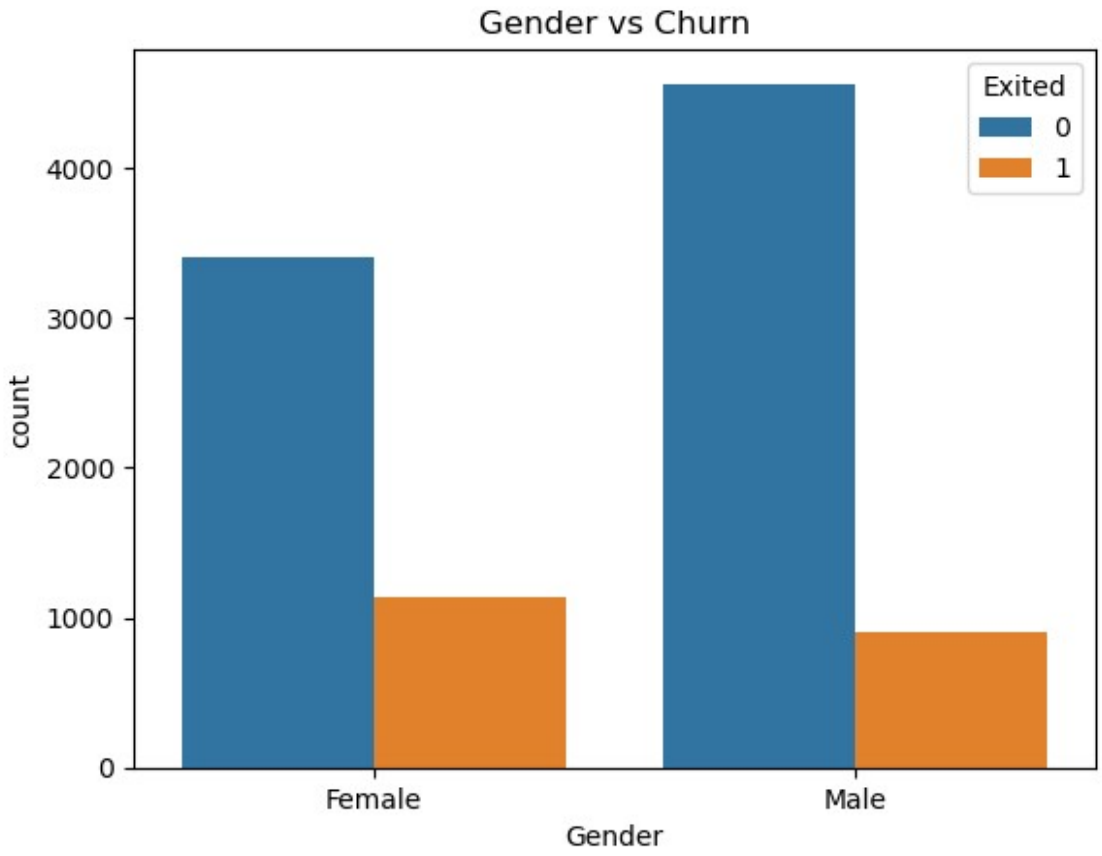
```python
# Create churn count graph
sns.countplot(x='Exited', data=df)

# Add title
plt.title("Customer Churn Count")

# Show graph
plt.show()
```



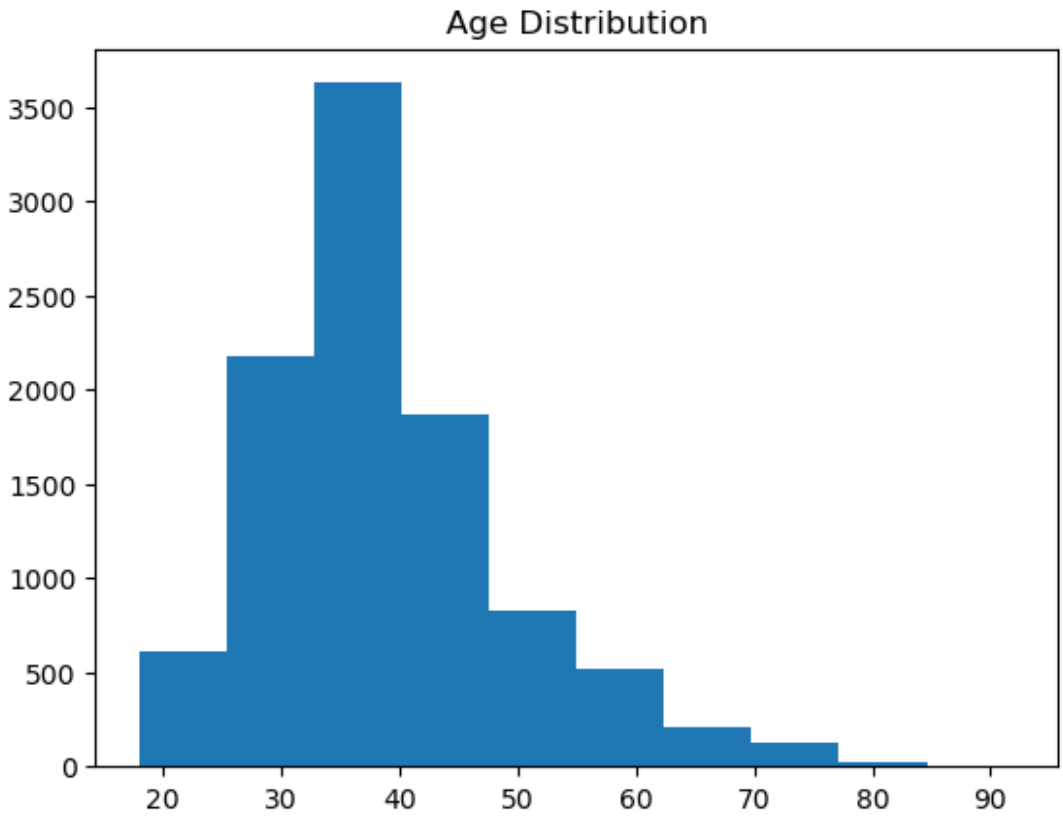
```
# Country-wise churn graph
sns.countplot(x='Geography', hue='Exited', data=df)

# Add title
plt.title("Country vs Churn")

# Show graph
plt.show()
```



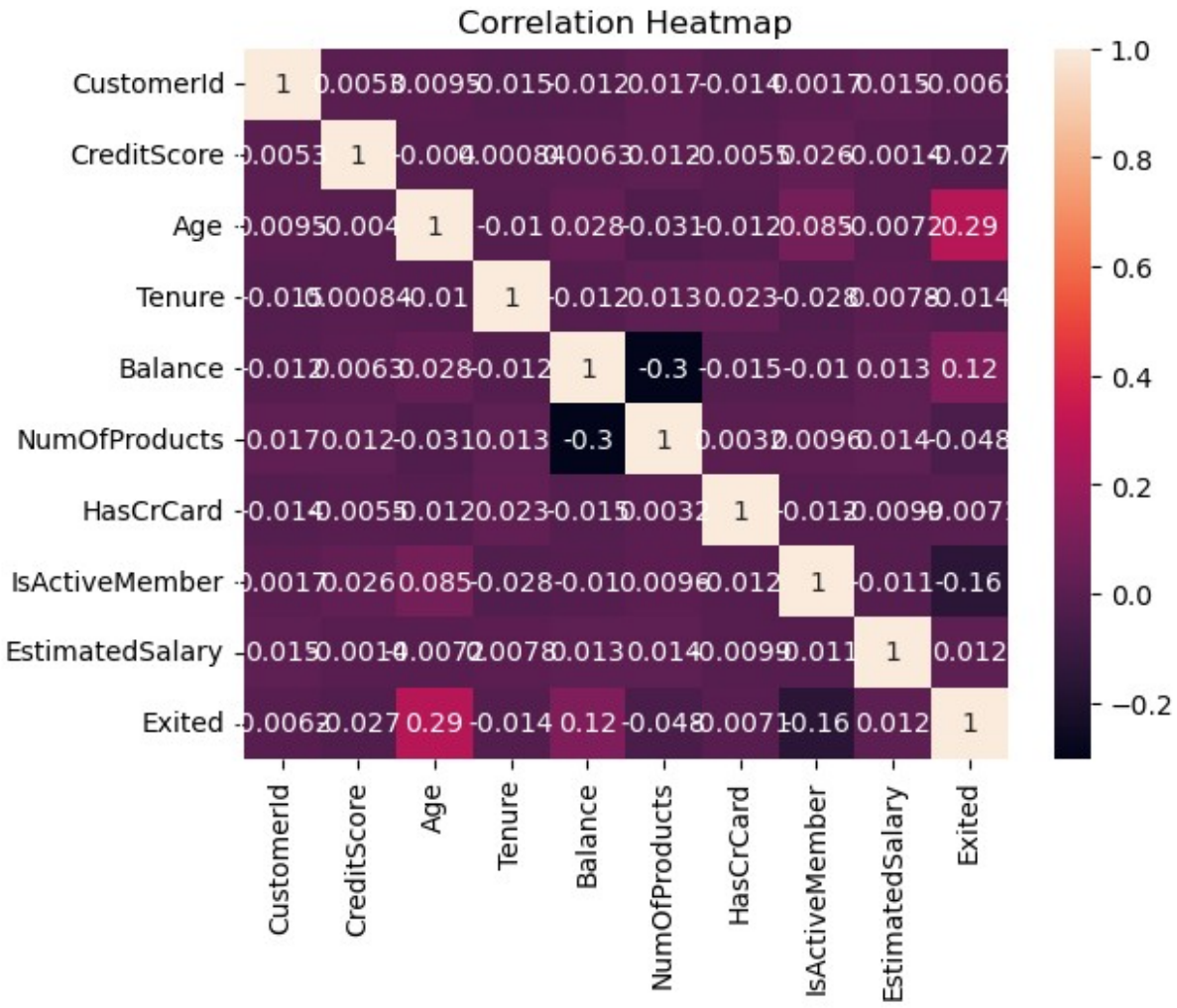

```
# Gender-wise churn graph
sns.countplot(x='Gender', hue='Exited', data=df)

# Add title
plt.title("Gender vs Churn")

# Show graph
plt.show()
```



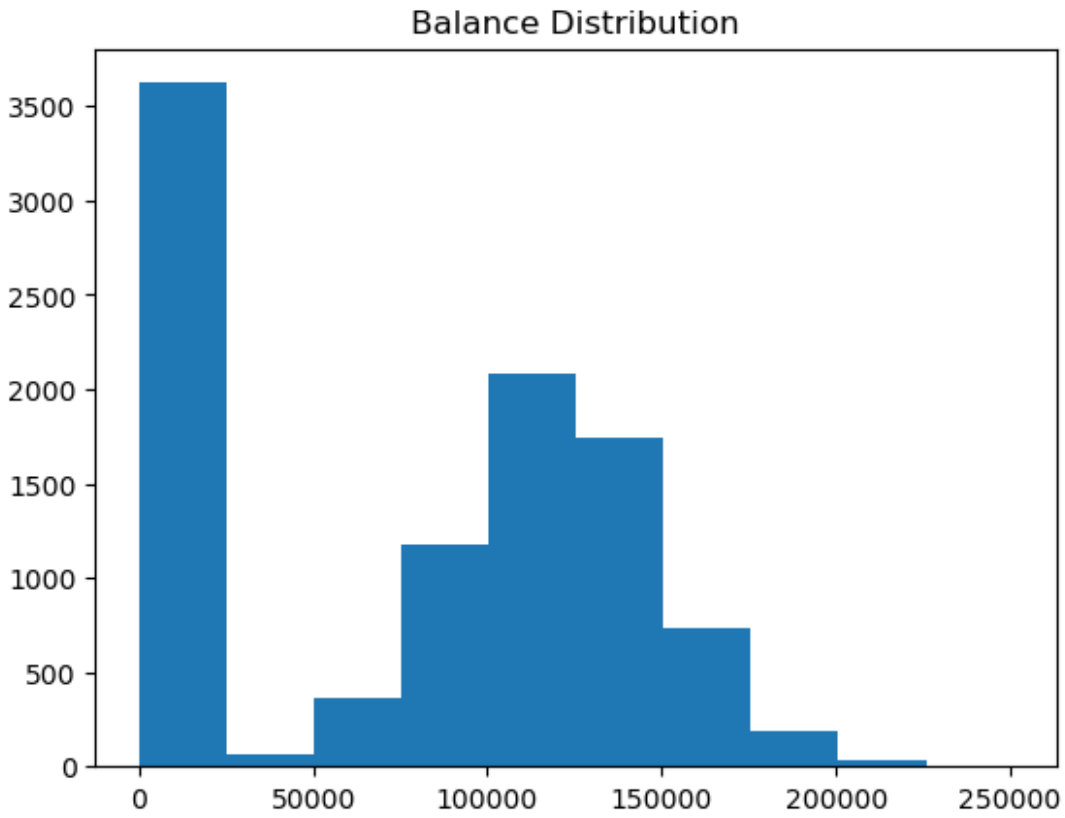

```
# Age histogram
plt.hist(df['Age'])

# Add title
plt.title("Age Distribution")

# Show graph
plt.show()
```



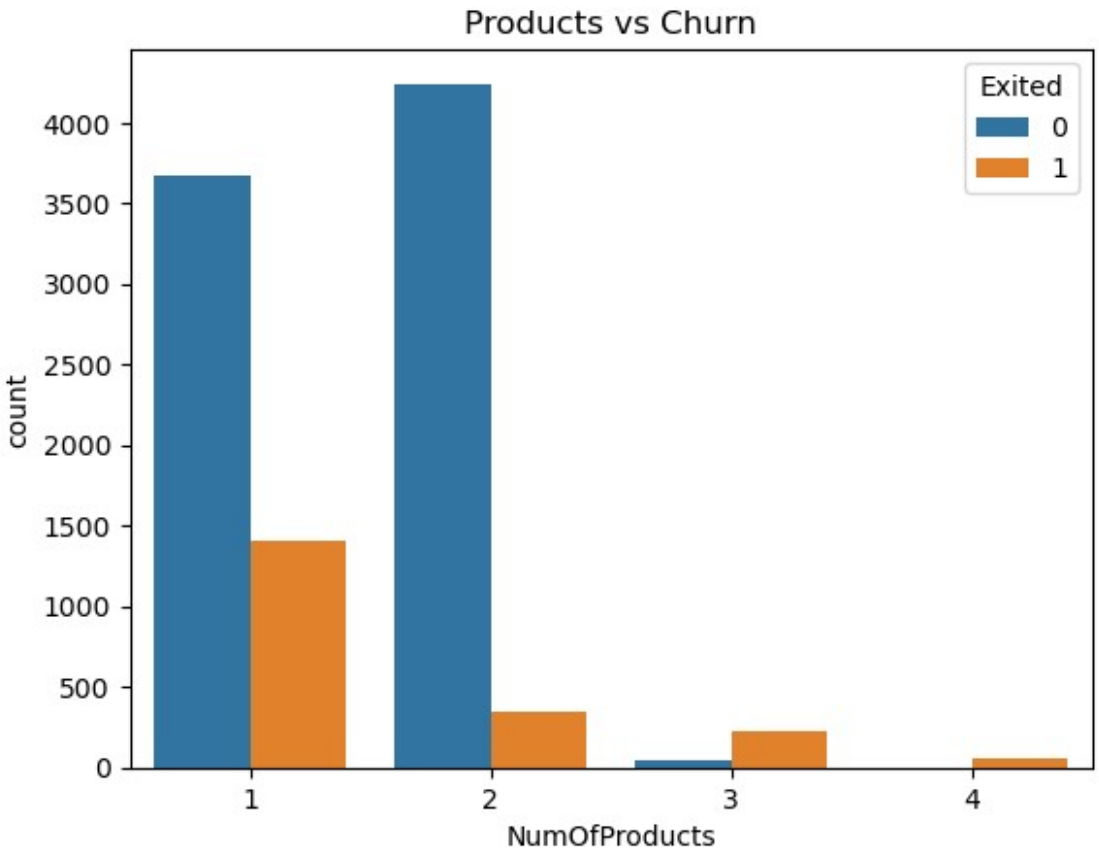

```
# Correlation matrix
corr = df.corr(numeric_only=True)

# Heatmap graph
sns.heatmap(corr, annot=True)

# Add title
plt.title("Correlation Heatmap")

# Show graph
plt.show()
```



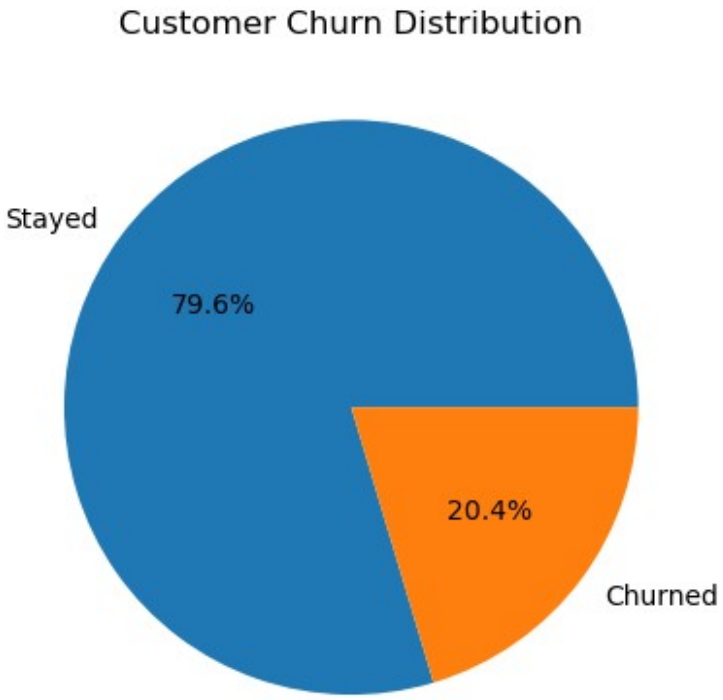

```
# Create balance histogram
plt.hist(df['Balance'])

# Add title
plt.title("Balance Distribution")

# Show graph
plt.show()
```




```
# Product-wise churn graph
sns.countplot(x='NumOfProducts', hue='Exited', data=df)

# Add title
plt.title("Products vs Churn")

# Show graph
plt.show()
```

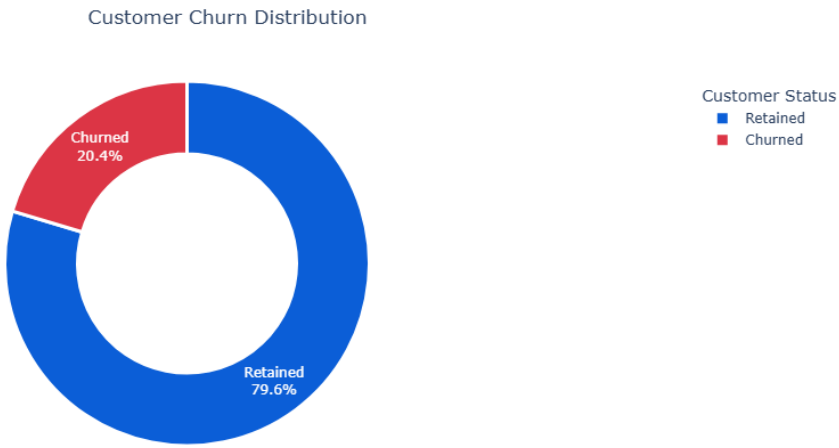



```
# Count churn values
churn_counts = df['Exited'].value_counts()

# Create pie chart
plt.pie(
    churn_counts,
    labels=['Stayed','Churned'],
    autopct='%1.1f%%'
)

# Add title
plt.title("Customer Churn Distribution")

# Show graph
plt.show()
```



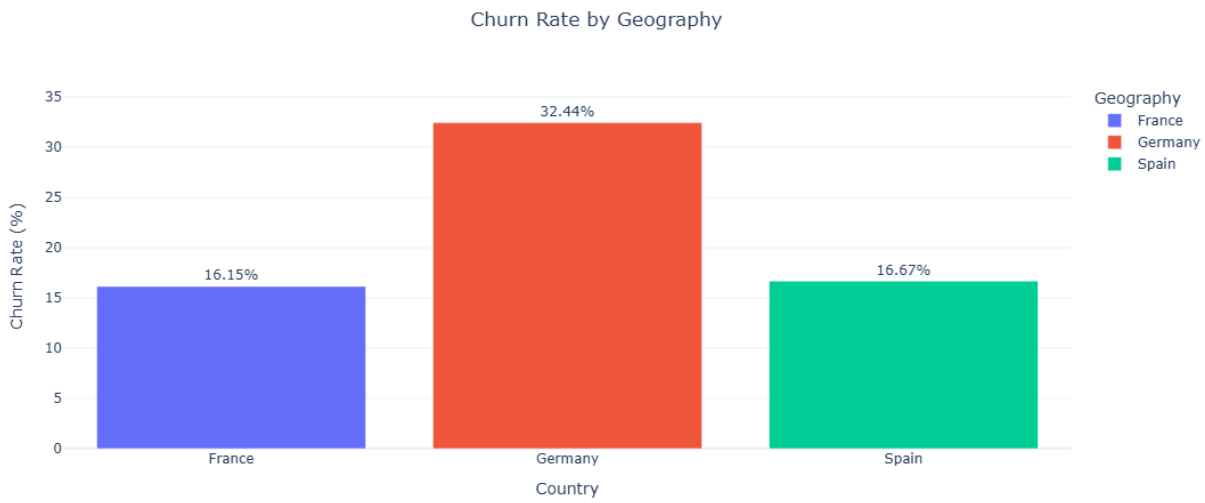

```
import plotly.express as px
import plotly.graph_objects as go

# Customer Counts
labels = ['Retained', 'Churned']
values = [retained_customers, churned_customers]

fig = go.Figure(
    data=[
        go.Pie(
            labels=labels,
            values=values,
            hole=0.60,
            textinfo='percent+label'
        )
    ]
)

fig.update_traces(
    marker=dict(
        colors=['#0B5ED7','#DC3545'],
        line=dict(color='white', width=3)
    )
)

fig.update_layout(
```

```
    title={
        'text':'Customer Churn Distribution',
        'x':0.5,
        'xanchor':'center'
    },
    legend_title="Customer Status",
    template='plotly_white',
    width=700,
    height=500
)

fig.show()
```



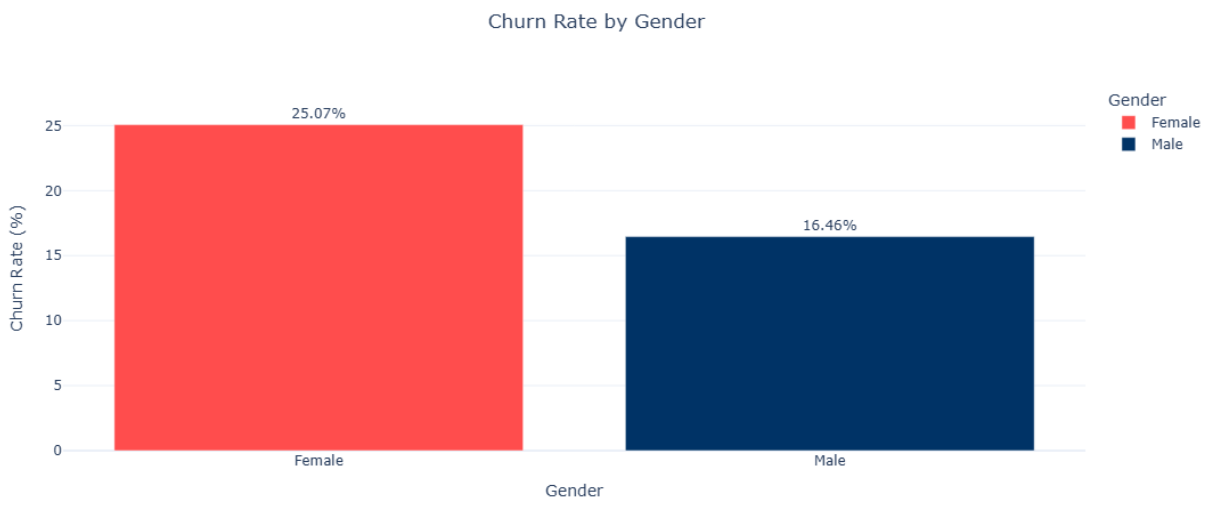

```
import plotly.express as px

# Geography-wise churn rate
geo_churn = (
    df.groupby("Geography")["Exited"]
      .mean()
      .reset_index()
)

# Convert to %
geo_churn["Exited"] = geo_churn["Exited"] * 100

# Plotly Bar Chart
fig = px.bar(
    geo_churn,
    x="Geography",
    y="Exited",
    color="Geography",
    text=geo_churn["Exited"].round(2),
```

```
    title="Churn Rate by Geography"
)

fig.update_traces(
    texttemplate='%{text:.2f}%',
    textposition='outside'
)

fig.update_layout(
    template="plotly_white",
    height=500,
    width=800,
    yaxis_title="Churn Rate (%)",
    xaxis_title="Country",
    title_x=0.5,
    legend_title="Geography"
)

fig.show()
```



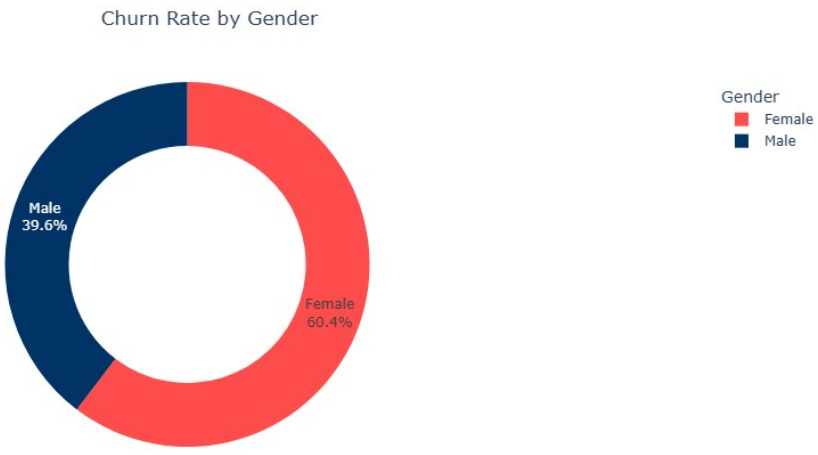

```
import plotly.express as px

gender_churn = (
    df.groupby("Gender")["Exited"]
      .mean()
      .reset_index()
)

gender_churn["Exited"] = round(
    gender_churn["Exited"] * 100,
    2
)
```

```
fig = px.bar(
    gender_churn,
    x="Gender",
    y="Exited",
    color="Gender",
    text="Exited",
    title="Churn Rate by Gender",
    color_discrete_map={
        "Male": "#003366",
        "Female": "#FF4D4D"
    }
)

fig.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig.update_layout(
    template="plotly_white",
    title_x=0.5,
    yaxis_title="Churn Rate (%)",
    height=500
)

fig.show()
```




```
import plotly.graph_objects as go

fig = go.Figure(
    data=[
```

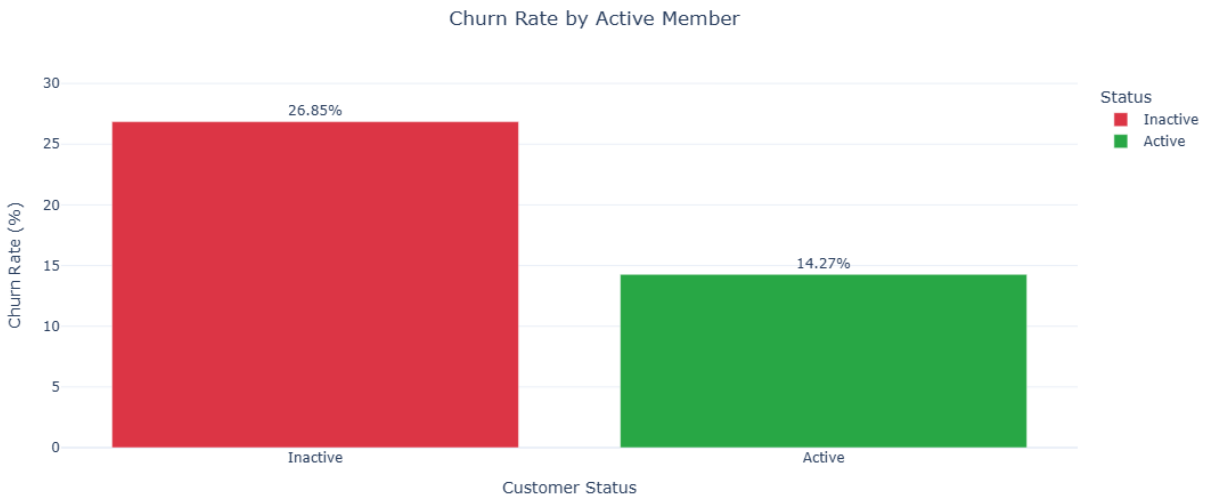
```
        go.Pie(
            labels=["Male", "Female"],
            values=[male_rate, female_rate],
            hole=0.65,
            textinfo="label+percent",
            marker=dict(
                colors=["#003366", "#FF4D4D"]
            )
        )
    ]
)

fig.update_layout(
    title={
        "text": "Churn Rate by Gender",
        "x": 0.5
    },
    template="plotly_white",
    width=700,
    height=500,
    legend_title="Gender"
)

fig.show()
```




```
# Create Active vs Inactive Churn Data

active_churn = (
    df.groupby("IsActiveMember")["Exited"]
      .mean()
      .reset_index()
)
```

```
# Convert to Percentage
active_churn["Exited"] = round(
    active_churn["Exited"] * 100,
    2
)

# Rename Values
active_churn["Status"] = active_churn["IsActiveMember"].replace({
    0: "Inactive",
    1: "Active"
})

active_churn
```

	IsActiveMember	Exited	Status
0	0	26.85	Inactive
1	1	14.27	Active



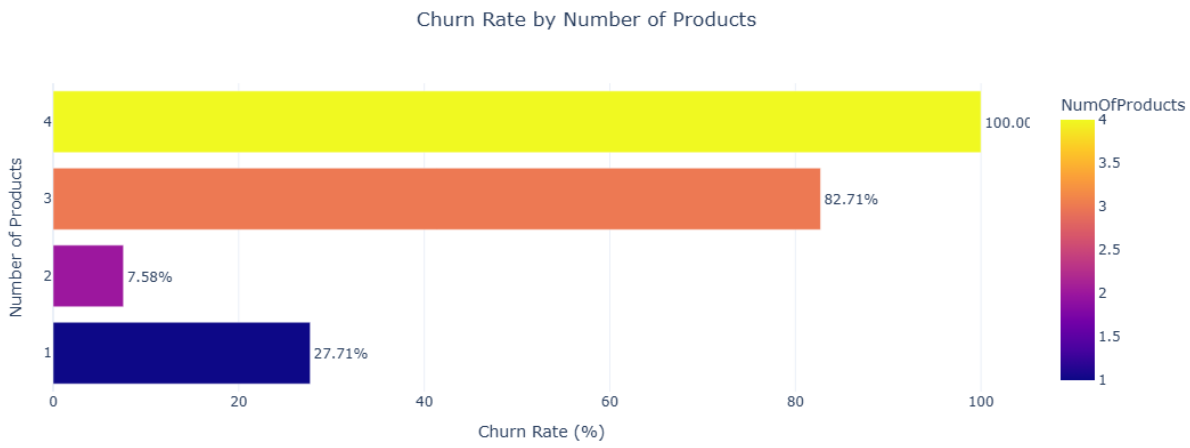

```
# Create Data
active_churn = (
    df.groupby("IsActiveMember")["Exited"]
      .mean()
      .reset_index()
)

active_churn["Exited"] = active_churn["Exited"] * 100

active_churn["Status"] = active_churn["IsActiveMember"].map({
    0:"Inactive",
```

```
    1:"Active"
})

# Churn Rate by Number of Products

product_churn = (
    df.groupby("NumOfProducts")["Exited"]
      .mean()
      .reset_index()
)

product_churn["Exited"] = round(
    product_churn["Exited"] * 100,
    2
)

product_churn
```

	NumOfProducts	Exited
0	1	27.71
1	2	7.58
2	3	82.71
3	4	100.00

```
import plotly.express as px

fig = px.bar(
    product_churn,
    x="Exited",
    y="NumOfProducts",
    orientation="h",
    color="NumOfProducts",
    text="Exited",
    title="Churn Rate by Number of Products"
)

fig.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig.update_layout(
    template="plotly_white",
    width=900,
    height=450,
    title_x=0.5,
    xaxis_title="Churn Rate (%)",
    yaxis_title="Number of Products",
    showlegend=False
)
```

```
fig.show()
```


Churn Rate by Number of Products
Number of Products
4
3
2
1
100.0(
82.71%
7.58%
27.71%
0
20
40
60
80
100
Churn Rate (%)
NumOfProducts
4
3.5
3
2.5
2
1.5
1